

Protocol-Governed Systems: A Conceptual Model

(c) 2026 Bhash Ganti

Bhash Ganti Contact: bachipeachy@gmail.com

Purpose: Define the conceptual model for Protocol-Governed Systems, validated through the PGS reference implementation.

Audience: Protocol designers, compiler authors, runtime implementers, conformance engineers

Abstract

Protocol-Governed Systems (PGS) propose a computational architecture in which governance precedes execution. Instead of relying on runtime policies, conventions, or post-hoc validation, PGS defines admissible behavior through governed protocol artifacts that are compiled into deterministic execution structures before runtime begins.

This document defines the conceptual model underlying PGS-Protocol. It establishes PGS as a constitutional execution substrate rather than a single application protocol. The model introduces the protocol snapshot as the immutable admissibility boundary between governance and execution, defines a compositional admissibility model spanning compile-time and runtime concerns, formalizes constitutional invariants governing execution behavior, and distinguishes protocol-defined semantics from implementation-defined mechanics.

The architecture separates governance from execution through a semantic-agnostic runtime constrained entirely by compiled admissibility structures. Execution produces immutable evidence projections (traces) that provide governance provenance without participating in admissibility decisions themselves.

The conceptual model has been validated through the PGS reference implementation (OmniBachi v0.2.0), an open-source ecosystem demonstrating deterministic execution, bounded authority, compile-time admissibility construction, and trace-backed execution evidence.

This document provides the semantic foundation for future work including conformance testing, AI governance evidence frameworks, and eventual formal protocol specification.

Preface

This document crystallizes the semantic kernel of Protocol-Governed Systems. It defines the foundational concepts, invariants, and boundaries that constitute PGS-Protocol — establishing the precise semantic model that the reference implementation validates and that all future conformant implementations must satisfy.

The foundational definitions this document establishes:

1. **What is PGS fundamentally?** — Define PGS as a constitutional execution substrate.
2. **What is a protocol snapshot?** — Define the immutable admissibility boundary between governance and execution.
3. **What is the unit of admissibility?** — Define precisely what is admitted and when.
4. **What are the constitutional invariants?** — State the axioms, not the guidelines.
5. **What is the minimal constitutional kernel?** — Define the smallest set of properties required for conformance.
6. **What is implementation-defined?** — Draw the boundary between protocol semantics and runtime mechanics.

Additional sections cover the evidence model as an orthogonal projection of execution, conformance philosophy, and deliberate deferrals — questions intentionally left open for resolution through future multi-implementation experience.

The conceptual model defined here serves as the prerequisite for a conformance test suite, an AI governance compliance evidence framework, a Yellow Paper formal specification, and eventual RFC standardization. Each of these subsequent bodies of work depends on the semantic precision established in this document.

1. What Is PGS Fundamentally?

1.1 Protocol or Substrate?

PGS-Protocol comprises several architecturally distinct layers: constitutional governance, a protocol artifact model, a compiler, a runtime, an evidence projection, and domain protocol definitions. Precise separation of these layers is essential — conflating them makes conformance impossible to define and interoperability impossible to achieve.

PGS governs admissible execution semantics rather than communication semantics. The central architectural question is whether PGS defines a single interaction model or a grammar within which multiple interaction models can be expressed.

A **protocol** defines one interaction model. HTTP defines hypermedia document exchange. SMTP defines mail exchange. There is one model per protocol.

A **substrate** defines the grammar within which multiple interaction models can be expressed. SQL is not one query — it is the language in which any relational query can be expressed [Codd, 1970]. LLVM IR is not one program — it is the substrate in which any compiled program can be represented.

PGS is a substrate.

The blockchain domain and the ai_governance domain are not two instances of the same protocol. They are two distinct governed execution protocols expressed using the same artifact grammar.

PGS-Protocol is therefore not analogous to HTTP. It is analogous to SQL: a governed execution substrate — the artifact grammar and execution semantics within which any PGS-conformant execution protocol is defined.

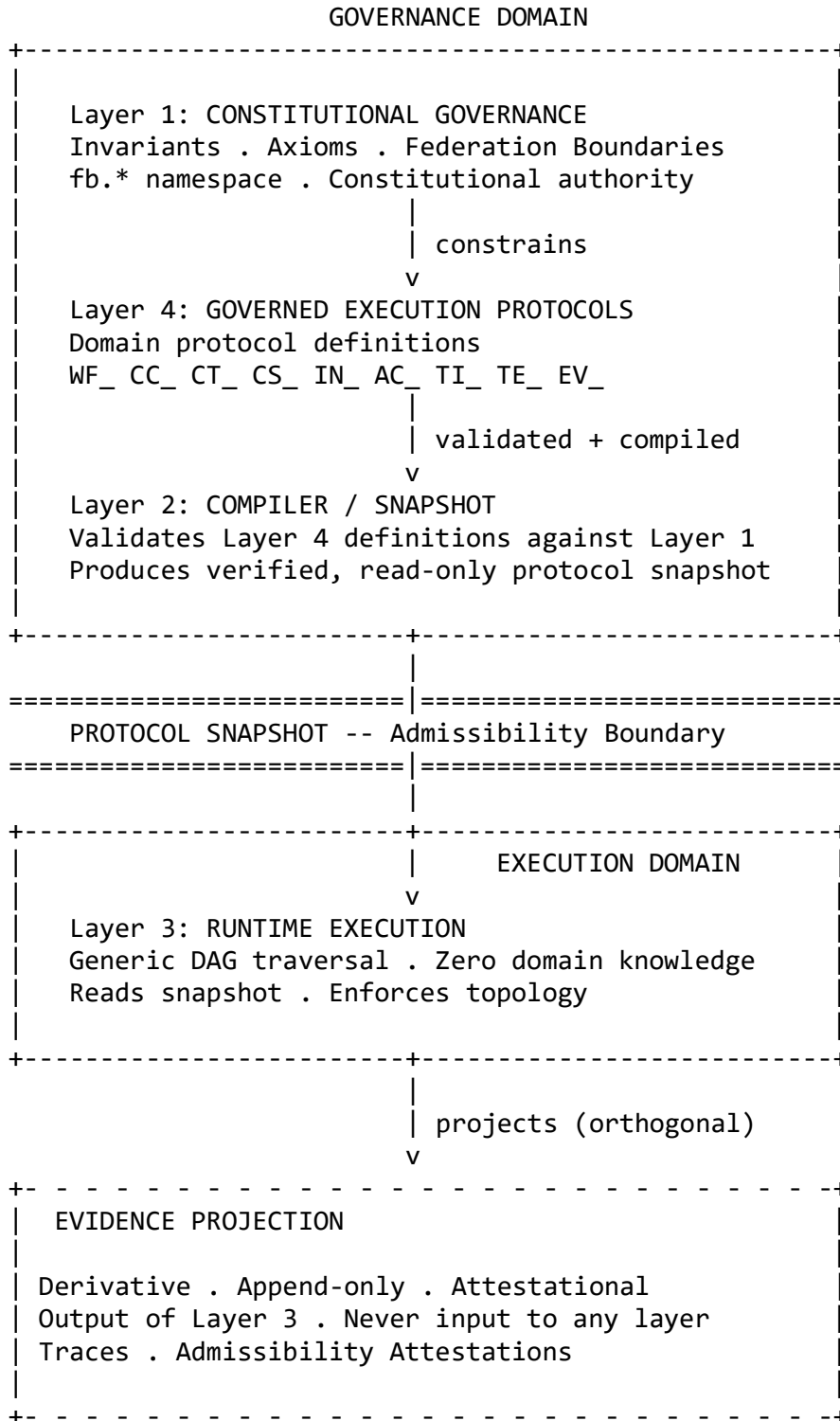
PGS is accordingly a durable execution grammar rather than a single interaction model. The substrate level endures because it governs the grammar of admissible orchestration rather than prescribing a specific protocol.

1.2 The Layered Model

PGS is composed of four execution layers plus an orthogonal evidence projection. Each layer has a defined role and a defined boundary.

A note on terminology: The term “layer” as used throughout this document is a contextual organizing concept for the PGS architectural model. It does not carry the semantics of layered architecture as defined in networking (e.g., the OSI model) or in layered software patterns (e.g., strict call-down hierarchies). PGS layers describe semantic dependency relationships — which concerns constrain which others — not communication stacks or runtime call chains. The numbering reflects dependency order, not execution sequence or encapsulation hierarchy.

Figure 1 — PGS Architectural Model



Evidence is not a peer layer. Evidence is an **orthogonal projection** — a derivative, append-only, observational output produced by the runtime during execution. Evidence does not govern execution, does not participate in admissibility, and does not feed back into any layer. It witnesses execution; it does not constitute it.

PGS-Protocol specifically names Layers 1 and 2 together: the constitutional governance layer and the artifact grammar / compilation contract that it governs. It is the meta-level — what makes a protocol definition *valid* and what a runtime must *do* to be conformant.

Layer numbering reflects semantic dependency order, not execution chronology. Layer 1 constrains Layer 4, which is compiled by Layer 2, which produces the snapshot consumed by Layer 3.

Layers 3 and 4 are implementation and instantiation — governed by PGS-Protocol but not constitutive of it.

1.3 Settled Definition

PGS-Protocol is the constitutional execution substrate — the governed artifact grammar and deterministic execution semantics within which any PGS-conformant execution protocol is defined, compiled, and executed.

It is:

- A protocol-definition substrate (not one specific protocol)
- Constitutional (invariants are non-negotiable axioms, not guidelines)
- Execution-oriented (governs what computation is admissible, not how messages are exchanged)
- Compile-time resolved (all structural behavior determined before execution)
- Semantically deterministic (identical inputs produce semantically equivalent outcomes)
- Session-less (each execution unit is atomic and self-contained; no implicit conversational execution continuity)
- Transport-agnostic (protocol semantics are independent of HTTP, CLI, gRPC, or any other transport)

Session-less does not mean stateless. The runtime may persist data across executions. Domain state (registries, event streams, governance records) is expected to exist and evolve over time. What session-less means is: no implicit execution context carries between invocations. Each execution unit receives its full context through its declared inputs — the snapshot, the actor context, and the payload. There is no ambient conversational state, no hidden session, no carryover from a prior invocation that is not explicitly declared as an input.

It is not:

- A workflow engine
- An orchestration framework
- A policy overlay
- A monitoring system
- An RPC protocol
- A messaging protocol
- A programming language
- A general-purpose computation substrate

PGS governs admissible orchestration of declared capabilities. The execution topology is finite, enumerable, and closed — not programmable. A conformant runtime possesses no domain knowledge. Domain semantics exist entirely within the admissibility plane of the snapshot.

1.4 The Inversion That Defines PGS

Existing systems execute behavior and then check it against policy.

PGS admits behavior before it executes.

This inversion is the conceptual core. It means governance is structural, not supervisory. Admissibility is a compile-time property, not a runtime check. Behavior is not an emergent property of code — it is a constructed property of protocol [Ganti, 2026a].

PGS constrains orchestration, not computation itself. The execution topology is governed; the implementations within that topology (CT_ and CS_) remain conventional code. This distinction is what separates a governed execution substrate from a programming language or general-purpose computation model.

2. The Protocol Snapshot

2.1 What a Snapshot Is

A **protocol snapshot** is the immutable admissibility boundary between governance and execution.

It is the compiled, verified, read-only artifact bundle that a runtime ingests. It contains all the information a generic runtime needs to execute any declared workflow — and nothing more.

The snapshot is:

- **Compiled** — produced by the compiler after validating Layer 4 protocol definitions against Layer 1 constitutional governance
- **Verified** — its existence is proof that constitutional admissibility (Section 3.2) and structural admissibility (Section 3.3) have been satisfied
- **Immutable** — frozen at compilation; the runtime may not modify it during or after execution
- **Self-contained** — all artifact references within the snapshot resolve within the snapshot; no external resolution at runtime
- **Versioned** — a snapshot version is an immutable semantic commitment; behavior change requires a new version

A snapshot is not a configuration file. It is not metadata. It is the **governing document** under which execution occurs — the constitutional artifact that defines what is admissible.

2.2 Snapshot as Admissibility Boundary

The snapshot is the single point at which governance meets execution:

- Everything **before** the snapshot (protocol source, compiler, constitutional governance) is the governance domain
- Everything **after** the snapshot (runtime, execution, traces) is the execution domain
- The snapshot itself is the **boundary artifact** — the one thing both domains agree on

The compiler writes the snapshot. The runtime reads it. Neither modifies it. This boundary is the structural guarantee that governance and execution remain separate concerns [Parnas, 1972].

2.3 Governance Surface and Execution Surface

Two concepts are essential for understanding what a snapshot governs:

Governance Surface — the complete set of admissible execution topologies enumerable from the snapshot. The governance surface is everything that *may* happen. It is fully defined at compile time and fully derivable from the snapshot. It is closed and finite.

Execution Surface — the subset of the governance surface that actually executes for a given (actor context, intent, payload) triple. The execution surface is everything that *does* happen in a specific invocation.

The relationship between these two surfaces is the central constraint of PGS:

The execution surface is always a subset of the governance surface. The runtime may only contract the governance surface (via authority admissibility), never expand it. No runtime mechanism — configuration, environment, plugin, extension — may admit execution behavior that the snapshot does not declare.

This is what it means for the runtime to be governed: it selects from a pre-declared set of admissible paths; it does not construct new ones.

2.4 Snapshot Contents

A protocol snapshot contains artifacts from two distinct planes:

Admissibility Plane — artifacts that govern what is permitted:

Artifact Type	Role
WF_	Workflow — execution topology DAG
CC_	Capability Contract — named DAG node with declared inputs, outputs, routing outcomes
CT_	Capability Transform — pure computation declaration
CS_	Capability Side Effect — controlled external state interaction declaration
IN_	Intent — declarative admission gate
AC_	Actor Context — authority binding
TI_	Transport Ingress — boundary normalization
TE_	Transport Egress — boundary projection
EV_	Event — control plane and observability signaling
Governance artifacts	Layers, invariants, assertions — compiled from fb.* federation boundaries

Realization Plane — artifacts that govern how permitted operations are concretely executed:

Artifact Type	Role
RB_	Runtime Binding — maps CT_/CS_ declarations to concrete implementations

All artifacts are addressed by FQDN: domain::ARTIFACT_CODE_Vn.

The separation of admissibility and realization planes within the snapshot is elaborated in Section 2.6.

2.5 Federation Boundary Semantics

A **federation boundary** is a constitutional sovereignty domain within which admissibility semantics are internally sovereign and externally declared.

Federation boundaries are not packaging units, deployment units, or organizational units. A federation boundary exists when and only when a distinct governance authority exists over a named set of protocol semantics. The creation of a federation boundary is triggered by the authorship of the first governance artifact (constitution, invariant, or assertion) that belongs within that boundary's authority.

Within a federation boundary:

- Protocol definitions are authored independently
- Constitutional governance applies to all definitions within the boundary
- The governance surface for that boundary’s artifacts is self-contained
- Admissibility is internally resolved under the boundary’s sovereignty

Across federation boundaries:

- Only explicitly declared interfaces are visible
- No federation boundary may assert authority over another
- Cross-boundary interaction requires explicit declaration on both sides
- Constitutional invariants (Section 4) apply universally; domain-specific admissibility is locally sovereign

Authority levels:

Federation boundaries operate at one of two authority levels:

- **Sovereign** — the root constitutional boundary (fb.constitution); source of all constitutional authority; governs the rules that other boundaries must satisfy
- **Delegated** — domain-specific boundaries (fb.topology, fb.transport, fb.authority, fb.vocabulary, fb.blockchain, fb.ai_governance, etc.) that operate under constitutional delegation from the sovereign boundary

The fb.* namespace governs the constitutional rules that apply to federation boundary definitions themselves — the rules about what makes a valid boundary, how boundaries declare their external interfaces, and what invariants all boundaries must satisfy.

Federation is not distribution. A single physical system may contain multiple federation boundaries. Multiple physical systems may implement a single federation boundary. Federation is about constitutional sovereignty scoping, not deployment topology.

2.6 The Realization Plane: RB_ Semantics

RB_ (Runtime Binding) occupies a unique position in the artifact model. It is the only artifact type that participates in **realization** but not **admissibility**.

RB_ answers: “Which concrete implementation fulfills this abstract capability declaration?”

RB_ does not answer: “Is this capability admissible?”

The distinction between the two planes within the snapshot:

Plane	Artifacts	Question Answered
Admissibility	AC_, IN_, WF_, CC_, CT_, CS_, TI_, TE_, EV_	What is permitted?
Realization	RB_	How are permitted operations concretely executed?

RB_ properties:

- **Resolved at startup** — all bindings are verified before any workflow execution; no dynamic binding discovery
- **Environment-scoped** — the same protocol may execute against different RB_ bindings in different environments (development vs. production) without changing what is admissible
- **Immutable during execution** — bindings cannot change after resolution
- **Template-parameterized** — RB_ artifacts may declare template parameters (e.g., `{{module_data_root}}`) that are substituted at binding resolution time, never discovered dynamically

A conformant runtime must ensure that RB_ resolution never influences:

- Authority decisions (AC_, IN_ evaluation)
- Topology traversal (WF_ DAG path selection)
- Admission gate evaluation (IN_ ACK/NACK)
- Governance surface enumeration (what the snapshot declares as admissible)

RB_ is a bridge between the protocol substrate and the implementation substrate. It exists in the snapshot (and is therefore snapshot-governed and snapshot-immutable) but it does not participate in the admissibility model.

Consequence: Two conformant runtimes with identical snapshots but different RB_ bindings will traverse identical topologies and produce semantically equivalent outcomes, provided the underlying CT_/CS_ implementations satisfy their declared contracts. The admissibility plane is identical; only the realization plane differs.

3. The Unit of Admissibility

3.1 Admissibility Is Compositional

There is no single atomic unit of admissibility. Admissibility operates at four layers, each with distinct timing and semantics.

This is intentional. It reflects the architecture: some questions can only be answered at compile time; others must be evaluated at runtime against the specific execution request.

3.2 Layer 1: Constitutional Admissibility

When evaluated: Compile time. **Question:** Does this protocol definition satisfy all constitutional invariants?

A protocol definition is constitutionally admissible if and only if it violates none of the axioms in Section 4. This is enforced by the compiler. A snapshot that exits compilation without error is constitutionally admissible.

Consequence: the runtime does not re-evaluate constitutional admissibility. If a snapshot exists, it is constitutionally clean.

3.3 Layer 2: Structural Admissibility

When evaluated: Compile time (verified at snapshot ingestion). **Question:** Are all artifacts referenced in this workflow DAG resolved within this snapshot?

A workflow execution is structurally admissible if:

- Every CC_ referenced in the WF_ DAG is present in the snapshot
- Every CT_ and CS_ declared by each CC_ is resolved via a valid RB_ binding
- Every IN_ referenced by the WF_ is present
- Every AC_ pattern referenced is present
- No circular DAG references exist
- All declared outcome routes lead to defined nodes

Structural admissibility is validated at compile time. The runtime verifies snapshot integrity on load; it does not re-resolve artifact references on each execution.

3.4 Layer 3: Authority Admissibility

When evaluated: Runtime, at the execution boundary (TI_ -> IN_). **Question:** Does this specific execution request satisfy the declared authority and admission conditions?

An execution request is authority-admissible if:

- The actor context (AC_) satisfies the authority requirements declared in the WF_ and IN_
- The payload satisfies all admission gate conditions declared in the IN_
- The request is directed to a workflow that exists in the loaded snapshot

Authority admissibility produces one of two outcomes: **ACK** (execution proceeds to WF_ traversal) or **NACK** (execution is rejected at the boundary; no DAG traversal occurs).

Authority admissibility is the mechanism by which the runtime contracts the governance surface to a specific execution surface. The runtime does not expand what is admissible — it selects from what the snapshot already permits.

3.5 Layer 4: Topological Admissibility

When evaluated: Runtime, per capability step. **Question:** Are this step's declared input requirements satisfiable from the current execution context?

Topological admissibility is guaranteed by structural admissibility (Layer 2) provided the DAG was correctly compiled. A runtime that has loaded a structurally admissible snapshot and received an authority-admissible request will not encounter unsatisfiable topological conditions — the compiler has guaranteed this.

Topological admissibility is therefore a runtime invariant check, not a runtime decision point.

3.6 The Composite Definition

An execution is PGS-admissible if and only if: 1. The snapshot is constitutionally admissible (Layer 1 — compile-time guarantee) 2. The workflow is structurally admissible within the snapshot (Layer 2 — compile-time guarantee, runtime-verified on load) 3. The execution request satisfies authority admissibility (Layer 3 — runtime, at TI/IN boundary) 4. All capability steps satisfy topological admissibility (Layer 4 — runtime invariant, guaranteed by Layer 2)

Layers 1 and 2 are compiler guarantees. Layers 3 and 4 are runtime enforcements. A runtime that receives a valid snapshot and evaluates Layer 3 correctly will never encounter a Layer 4 violation. Layer 4 violations in a running system indicate a corrupt or tampered snapshot.

3.7 The Execution Unit

An **execution unit** is a single admitted workflow traversal against a fixed snapshot under a single authority context and payload boundary.

It is the atomic unit of PGS execution. Every execution unit:

- Operates against exactly one snapshot version
- Has exactly one actor context (AC_)
- Has exactly one admitted intent and payload
- Traverses exactly one workflow topology (WF_ DAG)
- Produces exactly one execution witness (trace)
- Has a deterministic trace ID derivable from its inputs

The execution unit is the natural unit of:

- Admissibility evaluation (was this execution permitted?)
- Evidence production (what does the trace attest?)
- Deterministic replay (can this execution be independently reproduced?)

- Conformance testing (does the runtime behave correctly for this unit?)

3.8 What Is NOT an Admissibility Concern

The following are not admissibility concerns — they are execution outcomes:

- Whether a runtime side effect succeeds (CS_ failure -> VIOLATION outcome, not inadmissibility)
- Whether data already exists (ALREADY_EXISTS outcome, not inadmissibility)
- Whether an external system is reachable (BACKEND_ERROR outcome, not inadmissibility)

Admissibility is about whether execution is *permitted to proceed*. Outcomes describe what happened *during* permitted execution.

Additionally: **RB_ resolution is not an admissibility concern.** RB_ participates in realization, not admissibility (see Section 2.6). A change in runtime bindings does not alter what is admissible — it alters how admissible operations are concretely fulfilled.

4. Constitutional Invariants

These are axioms. They are not guidelines. They are not best practices. A system that violates any of these is not a PGS-conformant system — it is a PGS-inspired system, which is a different category.

4.1 Sovereignty Invariants

S1 — Protocol Sovereignty The protocol snapshot is the sole source of admissible execution behavior. No admissible execution behavior may originate outside the snapshot. The runtime may internally optimize, schedule, cache, or persist — but all admissible execution behavior is declared in the snapshot. No convention, environment variable, ambient configuration, or runtime assumption may introduce admissible behavior not declared in the snapshot.

S2 — Compile-Time Resolution All structural admissibility conditions are resolved before execution [Lamport, 2002]. The runtime may not synthesize, construct, modify, or infer execution topology. What is not in the snapshot does not execute.

S3 — No Ambient Authority All authority originates exclusively from declared (AC_, IN_, WF_, CC_) artifact chains. No authority may be asserted by role convention, runtime context, environment assumption, or any mechanism not declared in the snapshot [Saltzer and Schroeder, 1975].

S4 — Snapshot Immutability A compiled snapshot is read-only. The runtime may not modify it during or after execution. Snapshot content is frozen at compilation.

S5 — Version Immutability A version is an immutable semantic commitment. Behavior change requires a new version number. Versions are not overwritten, patched, or amended.

S6 — Runtime Non-Expansion The runtime cannot become more permissive than the snapshot. No runtime mechanism — configuration, environment, plugin, extension, or operator intervention — may admit execution behavior that the snapshot does not declare. The governance surface is set at compile time and may only contract at runtime (via authority admissibility), never expand. This is the heart of protocol sovereignty: the snapshot governs; the runtime obeys.

4.2 Execution Invariants

E1 — Semantic Determinism Identical admissible inputs (snapshot + actor context + payload) produce semantically equivalent execution outcomes and trace topology. Semantic determinism means: the execution path through the DAG, the capability step outcomes, the routing decisions, and the terminal result are identical. It does not mandate byte-level trace encoding identity across implementations — serialization ordering, internal timestamps, and storage formatting may vary. What may not vary is the semantic content: which steps executed, in what order, with what outcomes, producing what result.

E2 — CT Purity Capability Transforms (CT_) have zero side effects. CT_ may never invoke CS_. CT_ may never perform I/O, external calls, or state mutations. Purity is unconditional — there are no exceptions for “performance” or “convenience.” This invariant is grounded in the separation of pure computation from effectful interaction [Hughes, 1989; Moggi, 1991].

E3 — Fail Hard Missing artifact -> hard failure. Unresolvable input -> hard failure. There is no fallback, no silent degradation, no default behavior, no inference. The runtime surfaces a hard error or nothing at all.

E4 — No Topology Synthesis Execution topology is declared in the snapshot and resolved at compile time. The runtime may not construct, extend, or alter topology from payload content, environment state, or execution results.

E5 — Governance Before Execution Admissibility is determined before execution proceeds. The IN_ admission gate evaluates before any WF_ traversal begins. Behavior is not executed and validated afterward — it is admitted or rejected before traversal.

E6 — Admissibility Closure All admissible execution topology is fully derivable from the snapshot before execution begins. The governance surface (Section 2.3) is closed and enumerable. No execution path exists that cannot be traced to a declared topology in the snapshot. A conformant system can, given only the snapshot, enumerate every possible execution path without executing anything. This is one of the strongest differentiators of PGS and directly enables governance auditability, bounded execution verification, and evidence completeness.

4.3 Evidence Invariants

V1 — Trace Immutability Execution traces are output-only artifacts. Traces are never used as input to the compiler, runtime, or any other PGS component. Traces are append-only once written [Lamport, 1978].

V2 — Trace Completeness Every admitted execution (ACK from IN_) produces a complete trace. A partial trace represents a protocol violation (runtime failure during execution). The absence of a trace for an ACK'd execution is a protocol violation.

V3 — Trace Determinism For identical inputs, trace topology is semantically identical and trace IDs are identical. Trace IDs are derived deterministically from inputs, not from wall-clock time or random state. Semantic trace equivalence follows from E1 (Semantic Determinism).

4.4 Minimal Constitutional Kernel

The minimal constitutional kernel is the smallest set of capabilities a system must exhibit to be PGS-conformant. A system that satisfies the kernel is conformant regardless of its implementation complexity, deployment environment, or feature set.

The kernel comprises seven capabilities:

1. **Snapshot ingestion** — the system must load and verify a protocol snapshot, confirming structural admissibility (Section 3.3)
2. **Authority admission** — the system must evaluate the IN_ admission gate before any DAG traversal, producing ACK or NACK
3. **Topology traversal** — the system must execute the declared DAG in declared order with declared outcome routing; no topology synthesis
4. **CT purity enforcement** — the system must guarantee that CT_ executions produce zero side effects

5. **Evidence production** — the system must produce a complete execution witness (trace) for every admitted execution unit
6. **Snapshot immutability** — the system must not modify the snapshot during or after execution
7. **Semantic determinism** — identical admissible inputs must produce semantically equivalent outcomes

All constitutional invariants (Section 4.1-4.3) apply to any system exhibiting the kernel. The kernel defines the minimum capability; the invariants define the minimum correctness.

A runtime that implements the kernel in 200 lines and a runtime that implements it in 200,000 lines are equally conformant if both satisfy the invariants.

5. The Implementation Boundary

5.1 Why This Boundary Matters

The implementation boundary is the line between:

- **Protocol-defined:** semantic guarantees that any conformant implementation must provide
- **Implementation-defined:** mechanics that may vary across conformant implementations

Without this boundary clearly drawn, the protocol layer never stabilizes. Every implementation makes slightly different choices, and “conformance” becomes meaningless.

5.2 Protocol-Defined

The following are fixed for any PGS-conformant implementation:

Concern	Protocol Commitment
Artifact identity	FQDN format: domain::ARTIFACT_CODE_Vn. No short names. No version omission.
Nine execution concerns	TI, AC, IN, WF, CC, CT, CS, EV, TE — their semantic roles and interaction rules
Admissibility / realization separation	RB_ is realization-only; it does not participate in admissibility or confer authority
Constitutional invariants	All of Section 4 — non-negotiable
Admissibility model	All of Section 3 — four-layer compositional structure
Snapshot semantics	Immutable admissibility boundary; governance surface fully enumerable from snapshot
DAG execution model	Directed acyclic, outcome-routed, compile-time resolved
Input resolution mechanism	JSONPath-based resolution from prior step outputs (\$.results.<CC_CODE>.<field>)
Evidence semantics	Append-only, deterministic ID, output-only, complete per admitted execution unit
Federation namespace	fb.* as the constitutional governance namespace with sovereign/delegated authority levels
Compiler contract	A valid snapshot is one that has passed constitutional and structural admissibility validation
Governance/Execution surface relationship	Execution surface is always a subset of governance surface; runtime may only contract, never expand

Outcome vocabulary:

The protocol defines a structured outcome vocabulary organized by execution phase:

Phase	Outcome	Semantics
Admission	ACK	Execution request accepted; DAG traversal proceeds

Phase	Outcome	Semantics
Admission	NACK	Execution request rejected at admission gate; no traversal
Execution	SUCCESS	Admitted execution completed; capability produced expected result
Execution	VIOLATION	Admitted execution encountered a governed constraint violation during capability execution
Execution	ALREADY_EXISTS	Admitted execution detected an idempotency condition; no duplicate state produced
Execution	BACKEND_ERROR	Infrastructure or storage failure during capability execution; environmental, not constitutional

The outcome vocabulary is organized into two categories:

- **Admission outcomes** (ACK, NACK) — produced by the IN_ admission gate before DAG traversal. These are admissibility decisions.
- **Execution outcomes** (SUCCESS, VIOLATION, ALREADY_EXISTS, BACKEND_ERROR) — produced during DAG traversal by capability contracts. These are not admissibility decisions; they describe what happened during permitted execution.

Each CC_ declares its allowed outcome surface — the set of outcomes it may produce. The runtime routes on these outcomes; it does not interpret them. Outcome semantics are protocol-defined; outcome routing is snapshot-defined.

Open question (deferred): Whether the outcome vocabulary is closed (protocol-defined and exhaustive) or open (domain-extensible with protocol-defined categories) is deferred to the conformance suite.

5.3 Implementation-Defined

The following may vary across conformant implementations without violating protocol conformance:

Concern	Latitude
Transport binding	HTTP, CLI, gRPC, message queue, or any other; protocol is transport-agnostic
Serialization format	JSON is the reference implementation's choice; protocol mandates semantic structure, not encoding
Storage backend	How runtime data is persisted is an implementation concern
Runtime implementation language	Python is the reference; protocol is language-agnostic
Compiler implementation	Any compiler producing constitutionally admissible snapshots is protocol-conformant
Trace storage format	JSONL is the reference; protocol mandates evidence semantics, not file format

Concern	Latitude
Identity generation algorithms	Free within CT_ purity constraints (deterministic, no I/O)
RB_ lookup mechanism	How the runtime resolves runtime bindings is an implementation detail
Snapshot storage location	Protocol mandates read-only semantics; storage mechanism is implementation-defined
Internal runtime optimization	Scheduling, caching, persistence — free provided admissible behavior is unchanged

5.4 Deliberately Deferred

The following are not yet classified as protocol-defined or implementation-defined. They require resolution before a conformance suite can be written.

Open Question	Why Deferred
Canonical serialization	Should PGS-Protocol define a canonical wire format, or remain encoding-agnostic? Determines whether conformance includes byte-level representation.
Snapshot integrity model	Should snapshots carry cryptographic hash/signature as a first-class artifact? Determines whether admissibility attestations are externally verifiable.
Minimal conformant runtime	What is the minimum a runtime must implement to claim protocol conformance? Section 4.4 defines the kernel; the conformance suite operationalizes it.
Federation interop mechanics	How do two PGS systems federate at the network level? Semantics are defined (Section 2.5); mechanics require multiple implementations.

5.5 Conformance Philosophy

Conformance in PGS is **semantic, not implementation-identical**.

Two conformant implementations may differ in:

- Serialization format and encoding
- Storage backend and persistence strategy
- Runtime language and internal architecture
- Scheduling, caching, and optimization strategy
- Trace file format and storage layout
- RB_ binding targets (different concrete implementations of the same CT_/CS_ contracts)

Two conformant implementations may **not** differ in:

- Admissibility decisions — given identical snapshot and inputs, both must admit or reject identically
- Execution topology — given identical admitted inputs, both must traverse the same DAG path

- Outcome routing — given identical step outcomes, both must route to the same next step
- Terminal outcomes — given identical admitted inputs, both must produce semantically equivalent results
- Evidence completeness — both must produce complete traces for every admitted execution unit

Conformance is verified by the conformance suite against semantic equivalence, not by binary comparison of output artifacts.

6. The Evidence Model

Evidence is not an execution layer. It is an **orthogonal projection** — a derivative, append-only, observational output produced by the runtime during execution. Evidence does not govern execution. Evidence does not participate in admissibility. Evidence witnesses execution after the fact.

This section describes the evidence projection and its applications.

6.1 Traces Are Admissibility Attestations, Not Logs

The PGS trace is not a logging artifact. It is an **admissibility attestation** — a structured, immutable, deterministic record that a specific execution unit occurred within specific constitutional boundaries and was governed throughout.

The distinction matters:

- A log records what happened
- A witness records *that it was admissible to happen* and provides the evidence to verify that claim

Every trace contains:

Component	Content
Execution identity	Trace ID — deterministic from inputs
Admissibility record	Snapshot version, actor context, intent, payload hash
Topology traversal record	CC_ steps executed, in order, with outcomes
Input/output record	What each step received and produced
Terminal outcome	Final outcome and routing path taken
Constitutional provenance	Which snapshot version governed this execution

Together these constitute an **admissibility attestation**: a verifiable record that a specific execution unit was constitutionally governed.

6.2 What the Evidence Model Enables

The evidence model directly enables several capabilities that other approaches cannot provide:

Execution replay: Given the same snapshot and inputs, any conformant implementation must reproduce semantically equivalent trace topology (per E1 — Semantic Determinism).

Governance audit: The snapshot version at execution time is part of the evidence record. Auditors can verify not just what ran, but which constitutional version governed it.

Bounded authority proof: The actor context and admitted intent are part of the trace. The evidence record shows the authority chain that permitted the execution.

Deterministic verification: Because trace IDs are deterministic, a claim that “execution X occurred” is verifiable by replaying against the same inputs.

Governance surface enumeration: Because the governance surface is closed and enumerable (per E6 — Admissibility Closure), auditors can verify not only what did execute but what *could have* executed — the complete bounded execution space.

6.3 The AI Governance Compliance Application

Most AI governance systems today are policy overlays: they execute behavior and then check it against policy. The check happens after.

PGS inverts this. Governance is structural and pre-execution. The trace is not the governance mechanism — it is the evidence that governance occurred.

This maps directly to emerging regulatory requirements:

Regulatory Concern	PGS Evidence
Traceability of AI decisions	Trace: full topology traversal record
Authority provenance	AC_ + IN_ admissibility record in trace
Bounded capability demonstration	CC_ inputs/outputs, declared and evidenced; governance surface enumerability
Constitutional constraint proof	Snapshot version in trace; snapshot is the governing document
Deterministic replay	Trace ID determinism enables independent verification
Governance-before-execution	IN_ admission gate always precedes WF_ traversal
Bounded execution space	Governance surface is closed; all possible paths auditable from snapshot alone

PGS does not require post-hoc governance tooling to be layered on top. Governance provenance is a native output of every execution unit.

6.4 What the Evidence Model Does Not Currently Provide

The following are identified gaps in the evidence model as of v0.2.0:

- **Snapshot signing:** No cryptographic binding between a snapshot and its constitutional source. Adding this would make attestations externally verifiable without running the compiler.
- **Evidence portability:** Traces are currently JSONL files. There is no protocol-defined schema for consuming traces across implementations.
- **Cross-execution lineage:** No mechanism yet for linking traces across federated executions.

These gaps are noted, not resolved. Resolution depends on the snapshot integrity model (Section 5.4) and conformance suite development (Section 8.1).

7. Deliberate Deferrals

These questions are explicitly out of scope for this document. They are listed here so they are not accidentally introduced into early protocol discussions.

7.1 Wire Protocol and Serialization

PGS-Protocol does not currently define:

- A canonical binary or text encoding
- A protocol framing format
- Message envelope structure
- Header conventions

These are deferred until multiple implementations exist and serialization interoperability becomes an actual constraint.

7.2 Negotiation and Discovery

PGS-Protocol does not define:

- Protocol version negotiation
- Capability advertisement
- Federation discovery
- Trust negotiation
- Runtime advertisement

Session-less execution means negotiation semantics are orthogonal to the execution model. These are explicitly deferred.

7.3 Transport Bindings

The relationship between PGS-Protocol and specific transport substrates (HTTP, gRPC, message queues) is not specified. Transport bindings are implementation concerns. The protocol specifies execution semantics; binding to a transport is outside the protocol boundary.

7.4 Federation Interoperability Mechanics

How two PGS systems discover, authenticate, and interoperate with each other is deferred. Federation *semantics* (the `fb.*` namespace, sovereign/delegated authority levels, federation boundary declarations) are protocol-defined (Section 2.5). Federation *mechanics* (how federation actually operates at the network or runtime level) are deferred.

7.5 Hardware and Alternative Execution Environments

WASM execution, hardware execution, air-gapped sovereign deployments — these are valid long-term target environments. The protocol's transport-agnosticism and compile-time resolution are favorable to these environments. Specific bindings are deferred.

8. What This Document Unlocks

This document is not the endpoint. It is the prerequisite for four subsequent bodies of work:

8.1 Conformance Suite (direct next step)

With the constitutional invariants, admissibility model, and minimal constitutional kernel precisely defined, a conformance test suite can be written. The test suite is the operative protocol definition — executable semantics over prose semantics.

A runtime passes the conformance suite or it does not. A compiler produces conformant snapshots or it does not. This transforms the protocol from documentation into infrastructure.

8.2 AI Governance Compliance Evidence Framework (parallel track)

Section 6.3 maps PGS evidence directly to regulatory compliance requirements. The next step on this track is defining the evidence artifact structure precisely enough that auditors can consume it — without waiting for RFC formalization.

8.3 Yellow Paper (follows from conformance suite)

After the conformance suite exists, a Yellow Paper-style formal specification describes the protocol semantics in rigorous prose, with the test suite as the authoritative reference. This is the predecessor to any RFC.

8.4 RFC (deferred — requires multiple implementations)

The RFC is the correct artifact when multiple independent implementations exist, a conformance suite validates them, and the community needs a formal interoperability standard. Not before.

Appendix A: Glossary of Settled Terms

Term	Definition
PGS-Protocol	The constitutional execution substrate — the governed artifact grammar and deterministic execution semantics within which any PGS-conformant execution protocol is defined, compiled, and executed
Protocol snapshot	The immutable admissibility boundary between governance and execution; a compiled, verified, read-only artifact bundle that defines the governance surface
Governance surface	The complete set of admissible execution topologies enumerable from a snapshot; everything that <i>may</i> happen; closed and finite
Execution surface	The subset of the governance surface that actually executes for a given (actor context, intent, payload) triple; everything that <i>does</i> happen in a specific invocation
Execution unit	A single admitted workflow traversal against a fixed snapshot under a single authority context and payload boundary; the atomic unit of PGS execution
Federation boundary	A constitutional sovereignty domain within which admissibility semantics are internally sovereign and externally declared; exists when a distinct governance authority exists over a named set of protocol semantics
Admissibility plane	The set of snapshot artifacts (AC_, IN_, WF_, CC_, CT_, CS_, TI_, TE_, EV_) that govern what is permitted
Realization plane	The set of snapshot artifacts (RB_) that govern how permitted operations are concretely executed; does not participate in admissibility
Constitutional admissibility	Compliance of a protocol definition with all constitutional invariants; enforced by the compiler
Structural admissibility	Full DAG resolution within a single snapshot; all artifact references resolved at compile time
Authority admissibility	Runtime evaluation of whether a specific (actor, intent, payload) triple satisfies the admission gate for a given workflow; the mechanism by which governance surface contracts to execution surface
Execution witness	A trace — the immutable, deterministic record of an admitted execution unit, constituting an admissibility attestation
Admissibility attestation	The composite claim, evidenced by a trace and its governing snapshot, that a specific execution unit was constitutionally admissible
Evidence projection	The orthogonal, derivative, append-only output of execution; traces and attestations; never input to any PGS layer

Term	Definition
Protocol substrate	A governed artifact grammar within which multiple distinct governed execution protocols may be expressed — as distinguished from a protocol, which defines one interaction model
Constitutional invariant	An axiom that no conformant PGS system may violate; not a guideline or a best practice
Minimal constitutional kernel	The smallest set of capabilities (Section 4.4) a system must exhibit to be PGS-conformant
Protocol sovereignty	The principle that the compiled snapshot is the sole source of admissible execution behavior; no admissible behavior may originate outside it
Semantic determinism	The property that identical admissible inputs produce semantically equivalent execution outcomes and trace topology; does not mandate byte-level encoding identity
Admissibility closure	The property that all admissible execution topology is fully derivable from the snapshot before execution begins; the governance surface is closed and enumerable
Runtime non-expansion	The principle that the runtime may only contract the governance surface (via authority admissibility), never expand it; no runtime mechanism may admit behavior the snapshot does not declare

Appendix B: Relationship to PGS Reference Implementation

The PGS reference implementation (v0.2.0, OmniBachi runtime) is one conformant implementation of PGS-Protocol. It is not the definition of PGS-Protocol.

The reference implementation:

- Demonstrates constitutional admissibility (compiler validates)
- Demonstrates structural admissibility (snapshot ingestion validates)
- Enforces authority admissibility (TI -> IN boundary)
- Guarantees topological admissibility (DAG compiled by pgs_compiler)
- Produces execution witnesses (JSONL traces)
- Enforces CT purity (capability concern separation)
- Enforces snapshot immutability (read-only snapshot directory)
- Enforces runtime non-expansion (runtime reads snapshot; cannot modify governance surface)
- Separates admissibility and realization planes (RB_ resolved at startup, does not influence admissibility)

The reference implementation does not yet:

- Sign snapshots (Section 6.4 gap)
 - Produce portable evidence artifacts (Section 6.4 gap)
 - Include a conformance test harness (Section 8.1 next step)
 - Enumerate governance surface from snapshot (Section 2.3 capability; not yet tooled)
-

Appendix C: References

PGS Publications

Note: All referenced PGS publications are authored by the same author and serve as supporting materials for the conceptual model presented below.

Ganti, B. (2026a). Protocol-Governed Systems: A Constitutionally Constrained Architecture for Autonomous and AI-Generated Software. *Technical Paper*. DOI: [10.5281/zenodo.20272695](https://doi.org/10.5281/zenodo.20272695)

Ganti, B. (2026b). *Protocol-Governed Systems: A Practitioner's Guide — Version 0, First Edition*. DOI: [10.5281/zenodo.20278311](https://doi.org/10.5281/zenodo.20278311)

Ganti, B. (2026c). *Protocol-Governed Systems Field Manual v0*. DOI: [10.5281/zenodo.20278357](https://doi.org/10.5281/zenodo.20278357)

Reference Implementation

PGS Workspace v0.2.0 — eight-repository ecosystem, Apache-2.0 licensed. OmniBachi runtime published on PyPI. GitHub: [bachipeachy/pgs_workspace](https://github.com/bachipeachy/pgs_workspace)

Foundational Works

Codd, E.F. (1970). A relational model of data for large shared data banks. *Communications of the ACM*, 13(6):377-387.

Dijkstra, E.W. (1974). On the role of scientific thought. EWD447.

Hoare, C.A.R. (1969). An axiomatic basis for computer programming. *Communications of the ACM*, 12(10):576-580.

Programming Languages and Type Systems

Hughes, J. (1989). Why functional programming matters. *The Computer Journal*, 32(2):98-107.

Moggi, E. (1991). Notions of computation and monads. *Information and Computation*, 93(1):55-92.

Formal Methods and Specification

Lamport, L. (1978). Time, clocks, and the ordering of events in a distributed system. *Communications of the ACM*, 21(7):558-565.

Lamport, L. (2002). *Specifying Systems: The TLA+ Language and Tools for Hardware and Software Engineers*. Addison-Wesley.

Software Architecture and Design

Parnas, D.L. (1972). On the criteria to be used in decomposing systems into modules. *Communications of the ACM*, 15(12):1053-1058.

Security

Saltzer, J.H., and Schroeder, M.D. (1975). The protection of information in computer systems. *Proceedings of the IEEE*, 63(9):1278-1308.

Institutional Economics

North, D.C. (1990). *Institutions, Institutional Change and Economic Performance*. Cambridge University Press.

How to Cite

Ganti, B. (2026). Protocol-Governed Systems: A Conceptual Model. DOI: [pending Zenodo deposit]

Author Information

Bhash Ganti

Contact: bachipeachy@gmail.com ORCID: [0009-0007-3810-6520](https://orcid.org/0009-0007-3810-6520)

Document: Protocol-Governed Systems: A Conceptual Model (c) 2026 Bhash Ganti Predecessor to: PGS-Protocol Yellow Paper, PGS Conformance Suite